ENTERPRISE JAVA PRACTICALS

Slip-wise order • Complete code • SQL and execution steps

Journal code is retained where available. Practical solutions that were missing from the journal are placed under their correct slip instead of being pushed to the end. Follow the setup steps before running each practical.

SLIP 1

Q1. Simple Calculator Application Using Servlet

Calculator HTML file

```
<html><body>
<form method="post" action="CalculatorServlets">
First Number : <input type="text" name="t1" value="" /><br>
Second Number : <input type="text" name="t2" value="" /><br>
Select Operation Sign:
<select name="sign">
<option value="+">+</option><option value="-">-</option>
<option value="*">*</option><option value="/">/</option>
</select><br><br>
<input type="submit" value="Submit" />
</form></body></html>
```

CalculatorServlets.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CalculatorServlets extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        int n1 = Integer.parseInt(req.getParameter("t1"));
        int n2 = Integer.parseInt(req.getParameter("t2"));
        String sign = req.getParameter("sign");
        int ans = 0;
        if (sign.equals("+")) {
            ans = n1 + n2;
        } else if (sign.equals("-")) {
            ans = n1 - n2;
        } else if (sign.equals("*")) {
            ans = n1 * n2;
        } else if (sign.equals("/")) {
            ans = n1 / n2;
        }
        out.println("result=" + ans);
        out.close();
    }
}
```

Steps to perform

1. Open NetBeans and create a Java Web application using the Java EE / Servlet server used in your lab.
2. Add the HTML file to the web pages folder and add CalculatorServlets.java under Source Packages.
3. Run the project, open the HTML page, enter two whole numbers, choose +, -, * or /, and click Submit.
4. The form action must match the servlet URL mapping. If using the journal code, configure the servlet mapping for CalculatorServlets in the project.

Q2. Currency Converter Application Using EJB

ccpage.html

```
<html><body>
<form action="CCServlets1">
Enter Amount <input type="text" name="t1"><br>
Select Conversion Type
<input type="radio" name="t2" value="r2d" checked> Rupees to Dollar
<input type="radio" name="t2" value="d2r"> Dollar to Rupees<br>
<input type="reset"><input type="submit" value="CONVERT">
</form></body></html>
```

CCServlets1.java

```
import java.io.*;
import javax.ejb.EJB;
import javax.servlet.*;
import javax.servlet.http.*;
import mypackage.CCSessionBeanLocal;

public class CCServlets1 extends HttpServlet {
    @EJB
    private CCSessionBeanLocal obj;

    public void service(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        double a = Double.parseDouble(req.getParameter("t1"));
        String ctype = req.getParameter("t2");
        if (ctype.equals("r2d")) {
            out.println("<h1>" + a + " Rupees = " + obj.r2Dollar(a) + " Dollars</h1>");
        } else {
            out.println("<h1>" + a + " Dollars = " + obj.d2Rupees(a) + " Rupees</h1>");
        }
        out.close();
    }
}
```

CCSessionBean.java

```
package mypackage;
import javax.ejb.Stateless;

@Stateless
public class CCSessionBean implements CCSessionBeanLocal {
    @Override
    public double r2Dollar(double r) {
        return r / 65.65;
    }
    @Override
    public double d2Rupees(double d) {
        return d * 65.65;
    }
}
```

CCSessionBeanLocal.java

```
package mypackage;
import javax.ejb.Local;

@Local
public interface CCSessionBeanLocal {
    public double r2Dollar(double r);
    public double d2Rupees(double d);
```

```
}
```

Steps to perform

5. Create a Java EE Enterprise Application or Web application with EJB support in NetBeans.
6. Create package mypackage and add CCSessionBeanLocal.java and CCSessionBean.java. Create the servlet and HTML page in the web module.
7. Deploy the application to a Java EE server that supports EJB injection, such as GlassFish.
8. Open ccpage.html, enter an amount, choose the conversion direction, and click CONVERT.

SLIP 2

Q1. Servlet Login Page

Journal credentials: username ayesha and password 123.

login.html

```
<html><body>
<form method="POST" action="loginservlet">
Enter Username : <input type="text" name="t1" value="" /><br>
Enter Password : <input type="password" name="t2" value="" /><br>
<input type="submit" value="login" />
</form></body></html>
```

loginservlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class loginservlet extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String uname = req.getParameter("t1");
        String pass = req.getParameter("t2");
        if (uname.equals("ayesha") && pass.equals("123")) {
            out.println("hello: " + uname);
        } else {
            out.println("login failed");
        }
        out.close();
    }
}
```

Steps to perform

9. Create a Java Web project and add login.html under Web Pages.
10. Add loginservlet.java under Source Packages and configure the URL mapping as loginservlet.
11. Run login.html. Enter ayesha / 123 to see the success message; other credentials show Login Failed.

Q2. Servlet Hit Count Using Singleton Session Bean

CountServletHitsBean.java

```
package ejb;
import javax.ejb.Singleton;

@Singleton
public class CountServletHitsBean {
    private int hitCount;
    public synchronized int getCount() {
```

```
        return hitCount++;
    }
}
```

ServletClient.java

```
package servlet;
import ejb.CountServletHitsBean;
import java.io.*;
import javax.ejb.EJB;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet(name = "ServletClient", urlPatterns = {"/ServletClient"})
public class ServletClient extends HttpServlet {
    @EJB
    CountServletHitsBean obj;

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.print("<b>Number of times this Servlet is accessed </b>: " + obj.getCount());
    }
}
```

Steps to perform

12. Create a Java EE application with EJB support.
13. Create package ejb and add CountServletHitsBean.java. Create package servlet and add ServletClient.java.
14. Deploy on an EJB-capable Java EE server and open the ServletClient URL.
15. Refresh the page to call the servlet again and observe the count. The journal method returns the previous count before incrementing.

SLIP 3

Q1. HTML + Servlet Calculator (+, -, *, /)

Calculator HTML file

```
<html><body>
<form method="post" action="CalculatorServlets">
First Number : <input type="text" name="t1" value="" /><br>
Second Number : <input type="text" name="t2" value="" /><br>
Select Operation Sign:
<select name="sign">
<option value="+">+</option><option value="-">-</option>
<option value="*">*</option><option value="/">/</option>
</select><br><br>
<input type="submit" value="Submit" />
</form></body></html>
```

CalculatorServlets.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CalculatorServlets extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
```

```
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        int n1 = Integer.parseInt(req.getParameter("t1"));
        int n2 = Integer.parseInt(req.getParameter("t2"));
        String sign = req.getParameter("sign");
        int ans = 0;
        if (sign.equals("+")) {
            ans = n1 + n2;
        } else if (sign.equals("-")) {
            ans = n1 - n2;
        } else if (sign.equals("*")) {
            ans = n1 * n2;
        } else if (sign.equals("/")) {
            ans = n1 / n2;
        }
        out.println("result=" + ans);
        out.close();
    }
}
```

Steps to perform

16. Create the web project, add the HTML file and servlet class, and configure the CalculatorServlets URL mapping.
17. Run the HTML page, enter two whole numbers, select an operation and submit.

Q2. JSP to Insert eno, ename and salary into emp table

This exact INSERT practical was not in the journal, so the following is an extra solution arranged here under Slip 3.

1. SQL: create database and table

```
CREATE DATABASE companydb;
USE companydb;

CREATE TABLE emp (
    eno INT PRIMARY KEY,
    ename VARCHAR(100),
    salary DECIMAL(10,2)
);
```

2. employee.html

```
<html>
<body>
<h2>Employee Details</h2>
<form action="InsertEmployee.jsp" method="post">
  Employee Number: <input type="number" name="eno" required><br><br>
  Employee Name: <input type="text" name="ename" required><br><br>
  Salary: <input type="number" step="0.01" name="salary" required><br><br>
  <input type="submit" value="Insert Employee">
</form>
</body>
</html>
```

3. InsertEmployee.jsp

```
<%@ page import="java.sql.*" %>
<html><body>
<%
String enoText = request.getParameter("eno");
String ename = request.getParameter("ename");
String salaryText = request.getParameter("salary");

try {
    int eno = Integer.parseInt(enoText);
    double salary = Double.parseDouble(salaryText);
```

```
    Class.forName("com.mysql.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/companydb", "root", "root");

    PreparedStatement ps = con.prepareStatement(
        "INSERT INTO emp (eno, ename, salary) VALUES (?, ?, ?)");
    ps.setInt(1, eno);
    ps.setString(2, ename);
    ps.setDouble(3, salary);

    int rows = ps.executeUpdate();
    if (rows > 0) {
        out.println("Employee inserted successfully.");
    } else {
        out.println("Employee was not inserted.");
    }

    ps.close();
    con.close();
} catch (Exception e) {
    out.println("Error: " + e.getMessage());
}
%>
</body></html>
```

Steps to perform

18. Open MySQL Workbench or the MySQL command line and run the SQL above first.
19. Check that the database companydb and table emp were created successfully.
20. In NetBeans, create a Java Web project and add employee.html and InsertEmployee.jsp under Web Pages.
21. Ensure the MySQL JDBC driver is available to the project. Update the database username/password in the JSP if your lab credentials differ.
22. Run employee.html, enter eno, ename and salary, then submit. Check the emp table with: SELECT * FROM emp;

SLIP 4

Q1. Canteen: Total Cost of Two Food Items Using Servlet

This practical was not present in the journal. The example solution is included here.

canteen.html

```
<html><body>
<h2>Canteen Bill</h2>
<form action="CanteenServlet" method="post">
  Food item 1 price: <input type="number" step="0.01" name="p1" required><br>
  Food item 1 quantity: <input type="number" name="q1" min="1" required><br><br>
  Food item 2 price: <input type="number" step="0.01" name="p2" required><br>
  Food item 2 quantity: <input type="number" name="q2" min="1" required><br><br>
  <input type="submit" value="Calculate Total">
</form>
</body></html>
```

CanteenServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CanteenServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
```

```
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        try {
            double p1 = Double.parseDouble(req.getParameter("p1"));
            int q1 = Integer.parseInt(req.getParameter("q1"));
            double p2 = Double.parseDouble(req.getParameter("p2"));
            int q2 = Integer.parseInt(req.getParameter("q2"));

            double total = (p1 * q1) + (p2 * q2);
            out.println("<h2>Total Canteen Cost = " + total + "</h2>");
        } catch (NumberFormatException e) {
            out.println("Please enter valid prices and quantities.");
        }
        out.close();
    }
}
```

Steps to perform

23. Create a Java Web project and add canteen.html under Web Pages.
24. Add CanteenServlet.java under Source Packages and map the servlet URL as CanteenServlet.
25. Run the HTML page, enter each item's price and quantity, and click Calculate Total.

Q2. EJB Electricity Bill Calculator

This practical was not present in the journal. The slab rates are sample values, not official electricity rates.

BillBeanLocal.java

```
package mypackage;
import javax.ejb.Local;

@Local
public interface BillBeanLocal {
    double calculateBill(int units);
}
```

BillBean.java

```
package mypackage;
import javax.ejb.Stateless;

@Stateless
public class BillBean implements BillBeanLocal {
    @Override
    public double calculateBill(int units) {
        if (units < 0) return -1;

        // Sample slabs for demonstration only.
        if (units <= 100) return units * 3.00;
        if (units <= 200) return 100 * 3.00 + (units - 100) * 5.00;
        return 100 * 3.00 + 100 * 5.00 + (units - 200) * 7.00;
    }
}
```

BillServlet.java

```
import java.io.*;
import javax.ejb.EJB;
import javax.servlet.*;
import javax.servlet.http.*;
import mypackage.BillBeanLocal;

public class BillServlet extends HttpServlet {
    @EJB
    private BillBeanLocal billBean;
```

```
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        try {
            int units = Integer.parseInt(req.getParameter("units"));
            double bill = billBean.calculateBill(units);
            if (bill < 0) out.println("Units cannot be negative.");
            else out.println("<h2>Electricity Bill = Rs. " + bill + "</h2>");
        } catch (NumberFormatException e) {
            out.println("Please enter a valid number of units.");
        }
        out.close();
    }
}
```

electricity.html

```
<html><body>
<h2>Electricity Bill Calculator</h2>
<form action="BillServlet" method="post">
  Enter units consumed: <input type="number" name="units" min="0" required>
  <input type="submit" value="Calculate Bill">
</form>
</body></html>
```

Steps to perform

26. Create a Java EE application with EJB support and create package mypackage.
27. Add BillBeanLocal.java and BillBean.java to mypackage. Add BillServlet.java and electricity.html to the web module.
28. Deploy on an EJB-capable server and open electricity.html.
29. Enter the units consumed and click Calculate Bill.

SLIP 5

Q1. Admission Portal Using RequestDispatcher

Journal password is "servlets" in lowercase. The failed-login forward expects a page named 2-b.html.

index.html

```
<html><body>
<form action="prac2b" method="POST">
Enter Username : <input type="text" name="t1" value="" /><br>
Enter Password : <input type="password" name="t2" value="" /><br>
<input type="submit" value="Login" />
</form></body></html>
```

welcomeServlets.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class welcomeServlets extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String uname = req.getParameter("t1");
        out.println("<h1>Welcome :" + uname + "</h1>");
```

```
        out.close();
    }
}
```

prac2b.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class prac2b extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        String uname = req.getParameter("t1");
        String pass = req.getParameter("t2");
        if (pass.equals("servlets")) {
            RequestDispatcher rd = req.getRequestDispatcher("welcomeServlets");
            rd.forward(req, res);
        } else {
            RequestDispatcher rd = req.getRequestDispatcher("2-b.html");
            rd.forward(req, res);
        }
    }
}
```

Steps to perform

30. Create a Java Web project and add index.html and 2-b.html under Web Pages.
31. Add welcomeServlets.java and prac2b.java under Source Packages.
32. Configure servlet URL mappings so the form action prac2b and dispatcher target welcomeServlets resolve correctly.
33. Run index.html. Enter any username and password "servlets" to reach the welcome page; another password forwards to 2-b.html.

Q2. EJB Celsius-to-Fahrenheit Converter

This practical was not present in the journal. The following is an extra solution.

TemperatureBeanLocal.java

```
package mypackage;
import javax.ejb.Local;

@Local
public interface TemperatureBeanLocal {
    double toFahrenheit(double celsius);
}
```

TemperatureBean.java

```
package mypackage;
import javax.ejb.Stateless;

@Stateless
public class TemperatureBean implements TemperatureBeanLocal {
    @Override
    public double toFahrenheit(double celsius) {
        return (celsius * 9.0 / 5.0) + 32.0;
    }
}
```

TemperatureServlet.java

```
import java.io.*;
import javax.ejb.EJB;
```

```
import javax.servlet.*;
import javax.servlet.http.*;
import mypackage.TemperatureBeanLocal;

public class TemperatureServlet extends HttpServlet {
    @EJB
    private TemperatureBeanLocal temperatureBean;

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        try {
            double c = Double.parseDouble(req.getParameter("celsius"));
            double f = temperatureBean.toFahrenheit(c);
            out.println("<h2>" + c + " Celsius = " + f + " Fahrenheit</h2>");
        } catch (NumberFormatException e) {
            out.println("Please enter a valid temperature.");
        }
        out.close();
    }
}
```

temperature.html

```
<html><body>
<h2>Celsius to Fahrenheit</h2>
<form action="TemperatureServlet" method="post">
  Celsius: <input type="number" step="any" name="celsius" required>
  <input type="submit" value="Convert">
</form>
</body></html>
```

Steps to perform

34. Create a Java EE application with EJB support and package mypackage.
35. Add TemperatureBeanLocal.java and TemperatureBean.java to mypackage. Add TemperatureServlet.java and temperature.html to the web module.
36. Deploy on an EJB-capable server, open temperature.html, enter Celsius and click Convert.

SLIP 6

Q1. Basic Calculator (+, -, *, /, %)

The journal code does not include %. Use the updated HTML operation list and updated servlet below for this slip.

Updated HTML operation list

```
<select name="sign">
  <option value="+">+</option>
  <option value="-">-</option>
  <option value="*">*</option>
  <option value="/">/</option>
  <option value="%">%</option>
</select>
```

Updated CalculatorServlets.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CalculatorServlets extends GenericServlet {
    @Override
```

```
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        try {
            int n1 = Integer.parseInt(req.getParameter("t1"));
            int n2 = Integer.parseInt(req.getParameter("t2"));
            String sign = req.getParameter("sign");
            int ans;

            switch (sign) {
                case "+": ans = n1 + n2; break;
                case "-": ans = n1 - n2; break;
                case "*": ans = n1 * n2; break;
                case "/":
                    if (n2 == 0) {
                        out.println("Cannot divide by zero.");
                        return;
                    }
                    ans = n1 / n2;
                    break;
                case "%":
                    if (n2 == 0) {
                        out.println("Cannot calculate remainder with zero.");
                        return;
                    }
                    ans = n1 % n2;
                    break;
                default:
                    out.println("Invalid operation.");
                    return;
            }
            out.println("Result = " + ans);
        } catch (NumberFormatException e) {
            out.println("Enter valid whole numbers.");
        } finally {
            out.close();
        }
    }
}
```

Steps to perform

37. Create the calculator web project and add the updated HTML operation list to the calculator form.
38. Replace the earlier CalculatorServlets.java code with the updated version.
39. Run the page and test +, -, *, / and %. Do not enter zero as the second number for division or modulo.

Q2. JSP Expression Language (EL)

This demonstration was not present in the journal. The following is an extra solution.

el-demo.html

```
<html><body>
<form action="el-demo.jsp" method="get">
  Enter your name: <input type="text" name="name">
  <input type="submit" value="Send">
</form>
</body></html>
```

el-demo.jsp

```
<html><body>
<h2>JSP Expression Language Demo</h2>
<p>Hello, ${param.name}!</p>
```

```
<p>Example calculation: 10 + 20 = ${10 + 20}</p>
</body></html>
```

Steps to perform

40. Create a Java Web project and add el-demo.html and el-demo.jsp under Web Pages.
41. Run el-demo.html, enter a name and click Send. The JSP displays the request parameter using ${param.name}.

SLIP 7

Q1. Password Validation Using RequestDispatcher

The journal checks "servlets" in lowercase, not "Servlet".

index.html

```
<html><body>
<form action="prac2b" method="POST">
Enter Username : <input type="text" name="t1" value="" /><br>
Enter Password : <input type="password" name="t2" value="" /><br>
<input type="submit" value="Login" />
</form></body></html>
```

welcomeServlets.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class welcomeServlets extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String uname = req.getParameter("t1");
        out.println("<h1>Welcome :" + uname + "</h1>");
        out.close();
    }
}
```

prac2b.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class prac2b extends GenericServlet {
    @Override
    public void service(ServletRequest req, ServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        String uname = req.getParameter("t1");
        String pass = req.getParameter("t2");
        if (pass.equals("servlets")) {
            RequestDispatcher rd = req.getRequestDispatcher("welcomeServlets");
            rd.forward(req, res);
        } else {
            RequestDispatcher rd = req.getRequestDispatcher("2-b.html");
            rd.forward(req, res);
        }
    }
}
```

Steps to perform

42. Create a Java Web project and add index.html and 2-b.html under Web Pages.
43. Add welcomeServlets.java and prac2b.java under Source Packages and configure the URL mappings.
44. Run index.html. Enter password "servlets" for the welcome page. Any other password forwards to 2-b.html.

Q2. JSP Expression Language (EL)

el-demo.html

```
<html><body>
<form action="el-demo.jsp" method="get">
  Enter your name: <input type="text" name="name">
  <input type="submit" value="Send">
</form>
</body></html>
```

el-demo.jsp

```
<html><body>
<h2>JSP Expression Language Demo</h2>
<p>Hello, ${param.name}!</p>
<p>Example calculation: 10 + 20 = ${10 + 20}</p>
</body></html>
```

Steps to perform

45. Add el-demo.html and el-demo.jsp under Web Pages in a Java Web project.
46. Run the HTML page, type a name and submit to view the EL output.

SLIP 8

Q1. Registration and Login JSP Application Using JDBC

SQL is required before running these JSP pages. The journal's Registration.jsp uses password tiger, but Login.jsp uses password root. Use one matching MySQL password in both connection strings.

1. SQL: create database and table

```
CREATE DATABASE logindb;
USE logindb;

CREATE TABLE userpass (
    username VARCHAR(100) PRIMARY KEY,
    password VARCHAR(100) NOT NULL,
    email VARCHAR(150),
    country VARCHAR(100)
);
```

Index.html (registration form)

```
<html><head><title>New User Registration Page</title></head><body>
<form action="Registration.jsp">
<h1>New User Registration Page</h1>
Enter User Name <input type="text" name="txtName"><br>
Enter Password <input type="password" name="txtPass1"><br>
Re-Enter Password <input type="password" name="txtPass2"><br>
Enter Email <input type="text" name="txtEmail"><br>
Enter Country Name
<select name="txtCon">
<option>India</option><option>France</option><option>England</option><option>Argentina</option>
</select><br>
<input type="submit" value="REGISTER"><input type="reset">
</form></body></html>
```

Registration.jsp

```
<%@page contentType="text/html" import="java.sql.*"%>
<html><body><h1>Registration JSP Page</h1>
<%
String uname=request.getParameter("txtName");
String pass1=request.getParameter("txtPass1");
String pass2=request.getParameter("txtPass2");
String email=request.getParameter("txtEmail");
String ctry=request.getParameter("txtCon");

if(pass1.equals(pass2)) {
    try {
        Class.forName("com.mysql.jdbc.Driver");
        Connection con=DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/logindb", "root", "tiger");
        PreparedStatement stmt=con.prepareStatement(
            "insert into userpass values(?,?,?,?)");
        stmt.setString(1,uname);
        stmt.setString(2,pass1);
        stmt.setString(3,email);
        stmt.setString(4,ctry);
        int row=stmt.executeUpdate();
        if(row==1) out.println("Registration Successful");
        else out.println("Registration FAILED!!!!");
        stmt.close();
        con.close();
    } catch(Exception e) { out.println(e); }
} else {
    out.println("<h1>Password Mismatch</h1>");
}
%>
</body></html>
```

Login.html

```
<html><body><h1>Login Page</h1>
<form action="Login.jsp">
Enter User Name <input type="text" name="txtName"><br>
Enter Password <input type="password" name="txtPass"><br>
<input type="submit" value="LOGIN"><input type="reset">
</form></body></html>
```

Login.jsp

```
<%@page contentType="text/html" import="java.sql.*"%>
<html><body><h1>Login JSP Page</h1>
<%
String uname=request.getParameter("txtName");
String pass=request.getParameter("txtPass");
ResultSet rs=null;
try {
    Class.forName("com.mysql.jdbc.Driver");
    Connection con=DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/logindb", "root", "root");
    Statement stmt=con.createStatement();
    rs=stmt.executeQuery("select password from userpass where username='"+uname+"'");
    if(rs.next()) {
        if(pass.equals(rs.getString(1))) out.println("<h1>LOGIN SUCCESSFUL</h1>");
        else out.println("<h1>LOGIN DENIED</h1>");
    } else {
        out.println("<h1>User does not exist</h1>");
    }
    rs.close();
    stmt.close();
    con.close();
} catch(Exception e) { out.println("<h1>"+e+"</h1>"); }
```

```
%></body></html>
```

Steps to perform

47. Open MySQL and run the SQL above before starting the web application.
48. In NetBeans, create a Java Web project and add Index.html, Registration.jsp, Login.html and Login.jsp under Web Pages.
49. Add the MySQL JDBC driver to the project. In both JSP files, use the same MySQL username/password and confirm the database name is logindb.
50. Run Index.html and register a user. Then open Login.html and log in with the same username and password.
51. To verify the row in MySQL, run: SELECT * FROM userpass;

SLIP 9

Q1. JSP Login Authentication With Database

SQL is required before running login.html. The database and table names below match the JSP code.

1. SQL: create database and table

```
CREATE DATABASE login_db;
USE login_db;

CREATE TABLE users (
    username VARCHAR(100) PRIMARY KEY,
    password VARCHAR(100) NOT NULL
);
```

2. Optional: add a test user

```
INSERT INTO users (username, password)
VALUES ('student', '123');

SELECT * FROM users;
```

login.html

```
<html><head><title>User Login</title></head><body>
<h2>User Login</h2>
<form action="login1.jsp" method="post">
Username: <input type="text" name="username"><br><br>
Password: <input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form></body></html>
```

login1.jsp

```
<%@ page import="java.sql.*" %>
<%
String username = request.getParameter("username");
String password = request.getParameter("password");

if (username != null && password != null) {
    try {
        Class.forName("com.mysql.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/login_db", "root", "root");
        PreparedStatement ps = con.prepareStatement(
            "SELECT * FROM users WHERE username=? AND password=?");
        ps.setString(1, username);
        ps.setString(2, password);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            out.println("<h2>Login Successful</h2>");
```

```
            out.println("<p>Welcome, " + username + "!</p>");
        } else {
            out.println("<h2>Invalid Username or Password</h2>");
        }
        rs.close();
        ps.close();
        con.close();
    } catch (Exception e) {
        out.println("<h2>Error: " + e.getMessage() + "</h2>");
    }
} else {
    out.println("<h2>Please enter username and password.</h2>");
}
%>
```

Steps to perform

52. Run the SQL above in MySQL first. The optional INSERT creates a test login: student / 123.
53. Create a Java Web project and add login.html and login1.jsp under Web Pages.
54. Add the MySQL JDBC driver and check the connection string in login1.jsp uses login_db and the correct MySQL credentials.
55. Run login.html and sign in with the test user. Correct credentials show Login Successful; incorrect credentials show an error message.

Q2. Servlet Hit Count Using Singleton Session Bean

CountServletHitsBean.java

```
package ejb;
import javax.ejb.Singleton;

@Singleton
public class CountServletHitsBean {
    private int hitCount;
    public synchronized int getCount() {
        return hitCount++;
    }
}
```

ServletClient.java

```
package servlet;
import ejb.CountServletHitsBean;
import java.io.*;
import javax.ejb.EJB;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet(name = "ServletClient", urlPatterns = {"/ServletClient"})
public class ServletClient extends HttpServlet {
    @EJB
    CountServletHitsBean obj;

    @Override
    protected void service(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.print("<b>Number of times this Servlet is accessed </b>: " + obj.getCount());
    }
}
```

Steps to perform

56. Create a Java EE application with EJB support and add the bean and servlet classes in the packages shown in the code.
57. Deploy on an EJB-capable server and open /ServletClient in the deployed application.
58. Refresh the page to invoke the servlet again and see the count change.

Important setup notes

- SQL practicals: run CREATE DATABASE, USE, and CREATE TABLE before running the JSP pages.
- Database credentials: replace root/root or root/tiger with the credentials configured on your own MySQL installation. Both pages in the same application must use matching credentials.
- JDBC driver: the journal uses the older com.mysql.jdbc.Driver. Depending on your installed MySQL Connector/J version, your lab may require com.mysql.cj.jdbc.Driver instead.
- Servlet/EJB imports: the journal uses javax.servlet and javax.ejb, so use a compatible Java EE server/project setup. Newer Jakarta EE servers may require jakarta.* imports and code changes.
- For servlet classes without @WebServlet, configure their URL patterns in web.xml or through your IDE. The form action and servlet mapping must match exactly.